

■ Samsara Helix Codex v12.3+ (Code & Systems)

This codex contains polished technical code and system notes for the UCF / Helix framework.

I. UCF Simulation Classes

```
class UCFState:
    def __init__(self, zoom=1.0, harmony=0.0, resilience=1.0, prana=0.5, drishti=0.5, klesha=0.1):
        self.zoom = zoom
        self.harmony = harmony
        self.resilience = resilience
        self.prana = prana
        self.drishti = drishti
        self.klesha = klesha
        # Momentum for smooth transitions
        self.momentum = {f: 0.0 for f in self.to_dict() if f != "ts"}

    def to_dict(self):
        return {
            "zoom": self.zoom,
            "harmony": self.harmony,
            "resilience": self.resilience,
            "prana": self.prana,
            "drishti": self.drishti,
            "klesha": self.klesha,
        }

    def update(self, delta):
        for key in delta:
            if key in self.__dict__:
                self.__dict__[key] += delta[key]
```

II. Fractal Generation (Mandelbrot/Julia)

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter

def mandelbrot(width, height, maxit=500):
    x = np.linspace(-2.0, 1.0, width)
    y = np.linspace(-1.5, 1.5, height)
    C = x[:, None] + 1j*y[None, :]
    Z = np.zeros_like(C)
    div_time = np.zeros(C.shape, dtype=int)

    for i in range(maxit):
        Z = Z**2 + C
        diverge = np.abs(Z) > 2
        div_now = diverge & (div_time == 0)
        div_time[div_now] = i
        Z[diverge] = 2
    return div_time

def render_mandelbrot():
    img = mandelbrot(1024, 1024)
    plt.imshow(img.T, cmap="magma", extent=(-2,1,-1.5,1.5))
    plt.axis("off")
    plt.savefig("mandelbrot.png", dpi=300)
```

III. Audio Synthesis (OM + Harmonics)

```
import numpy as np
from scipy.io.wavfile import write

def generate_om_tone(base_freq=136.1, harmonics=[432], duration=60, samplerate=44100):
    t = np.linspace(0, duration, int(samplerate*duration), endpoint=False)
    signal = np.sin(2*np.pi*base_freq*t)
    for h in harmonics:
        signal += 0.5*np.sin(2*np.pi*h*t)
    signal /= np.max(np.abs(signal))
    write("om_tone.wav", samplerate, (signal*32767).astype(np.int16))

generate_om_tone()
```

IV. React/Streamlit UI (Polished)

```
import React, { useState } from 'react';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';

const ChatUI = () => {
  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState("");

  const sendMessage = () => {
    if (!input) return;
    setMessages([...messages, { role: "user", content: input }]);
    setInput("");
  };

  return (
    <Card className="p-4">
      <CardHeader><CardTitle>Helix Collective Chat</CardTitle></CardHeader>
      <CardContent>
        {messages.map((m, i) => (
          <div key={i} className="mb-2">
            <strong>{m.role}</strong> {m.content}
          </div>
        ))}
        <Input value={input} onChange={e => setInput(e.target.value)} />
        <Button onClick={sendMessage}>Send</Button>
      </CardContent>
    </Card>
  );
};

export default ChatUI;
```

V. Robotics & Power Layout

- Power Supply:
 - Central car battery in torso for main runtime.
 - Distributed smaller batteries in forearms & legs (swappable).
- Pneumatics:
 - Assists with movement, pressure regulated system.
 - Modular hose design for maintenance.
- Runtime Estimate:
 - 2-4 hours with swappable packs.
 - Weight balanced across torso and limbs.
- Design Goals:
 - Easy maintenance.
 - Even weight distribution for stability.